EECS 482

Lab 9: Networking & File Systems

Administrivia

Today: 07/18

Project 3

Due 07/21

Project 4

Assigned 07/21

Agenda

1. Performance Metrics
2. Networking

Throughput vs Responsiveness

Definitions

- Throughput: The number of jobs done per timeframe
  - The number of HTTP requests per second
  - The number of SQL queries per second
  - The number of tokens/words given by an LLM per second
- Responsiveness: Time to completion for each job
  - Time the client waits for website to load
  - Time the server waits for SQL queries to process
  - Time the user waits for ChatGPT to finish responding
- A **response throughput graph** plots the response time against the throughput, useful to determine the optimal performance point of the system

Let's take some measurements

- Create some arbitrary task
  - Ex. Sum up 1 to 100000000
  - Run some variable number of threads at once
- Record the time

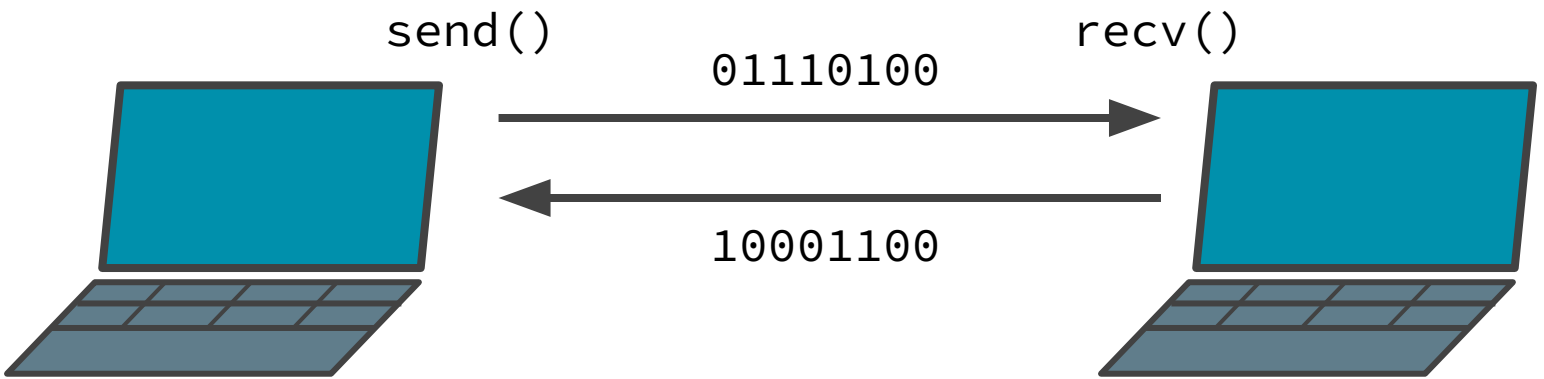
- Store (jobs/ time , avg response time) for each run
- Bonus: Graph it

https://os.eecs.umich.edu/lab/sockets.html

Sockets

Sockets: inter-process communication channels

Two-way byte stream between two processes or machines



Quick note: POSIX-style APIs

We don’t get to do:

```
Socket sock(...);
sock.send(...);
sock.recv(...);
...
```

Quick note: POSIX-style APIs

Typically, in C, it looks like this:

```
socket_t* sock = socket(...);
send(sock, ...);
recv(sock, ...);
...
```

Quick note: POSIX-style APIs

Actually, these OS apis use *file descriptors*, not pointers:

```
// Integer “handle” to os-internal socket object
int sockfd = socket(...);
send(sockfd, ...);
recv(sockfd, ...);
...
```

POSIX socket interface

```
int socket(domain, type, protocol);

int send(sockfd, buffer, size, flags);
int recv(sockfd, buffer, size, flags);

int bind   (sockfd, addr);
int listen (sockfd, backlog);
int accept (sockfd, 0, 0);
int connect(sockfd, addr);
int close  (sockfd);
```

Sending and receiving with sockets

```
// Send up to len bytes to peer from buffer.
// Returns number of bytes sent, or < 0 if error.
int send(int sockfd, void *buffer, size_t len, int flags);

// Receive up to len bytes from peer into buffer.
// Returns number of bytes received, or < 0 if error.
int recv(int sockfd, void *buffer, size_t len, int flags);
```

Sending and receiving with sockets

```
// Returns number of bytes sent, or < 0 if error.
int send(int sockfd, void *buffer, size_t len, int flags);

// Assume we already set up a socket and obtained sockfd
std::string message = “Hello, peer!”;

// Send message - send() may send fewer than the requested
// number of bytes - how would you accommodate this?
send(sockfd, message.data(), message.length(), 0);
```

Sending and receiving with sockets

```
/* Returns number of bytes received, or < 0 if error. */
int recv(int sockfd, void *buffer, size_t len, int flags);

// Assume we already set up a socket and obtained sockfd
std::string message;
char buf[64];

// Like send(), recv() is not guaranteed to send len bytes
int bytes_recvd = recv(sockfd, buf, sizeof(buf), 0);
message = std::string(buf, bytes_recvd);
```

Sending and receiving with sockets

```
/* Returns number of bytes received, or < 0 if error. */
int recv(int sockfd, void *buffer, size_t len, int flags);

// Assume we already set up a socket and obtained sockfd
std::string message;
char buf[64];

int bytes_recvd = recv(sockfd, buf, sizeof(buf), 0);
message = std::string(buf, bytes_recvd);
```

Note: recv() does not return 0 until client calls close()

Sending and receiving with sockets **(with std::array)**

```
/* Returns number of bytes received, or < 0 if error. */
int recv(int sockfd, void *buffer, size_t len, int flags);

// Assume we already set up a socket and obtained sockfd
std::string message;
std::array<char, 64> buf;

int recvd = recv(sockfd, buf.data(), buf.size(), 0);
message.append(buf.data(), recvd);
```

Sending and receiving with sockets

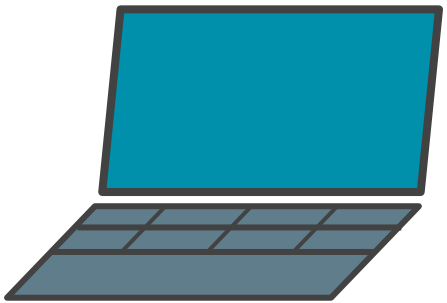
```
/* Returns number of bytes received, or < 0 if error. */
int recv(int sockfd, void *buffer, size_t len, int flags);
// Assume we already set up a socket and obtained sockfd

/* Handy recv flag: MSG_WAITALL
 * Causes recv to block until exactly len bytes recv'd. */

char UMID[8]; // Your UMID is always 8 digits
int recvd = recv(sockfd, UMID, 8, MSG_WAITALL);
assert(recvd == 8);
```

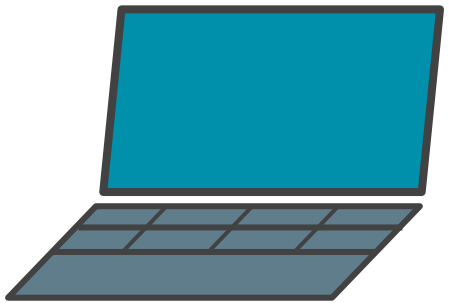
How to set up the connection?

Anybody
wanna talk?



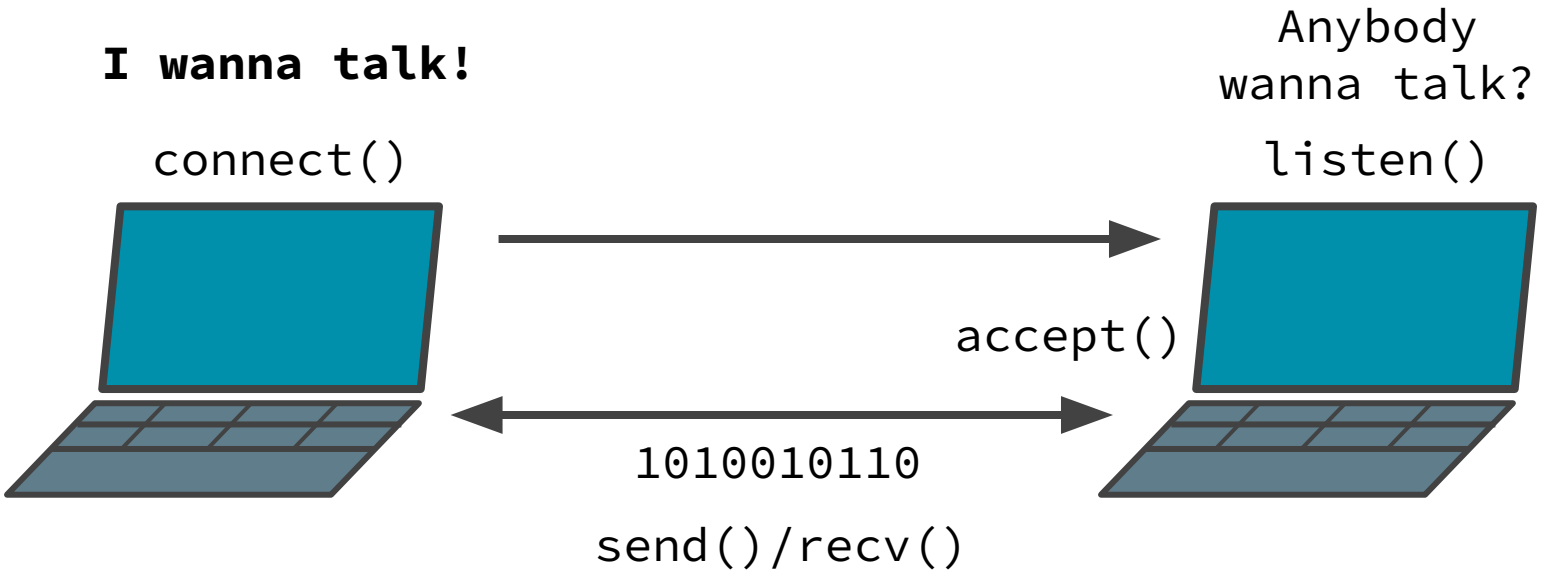
...

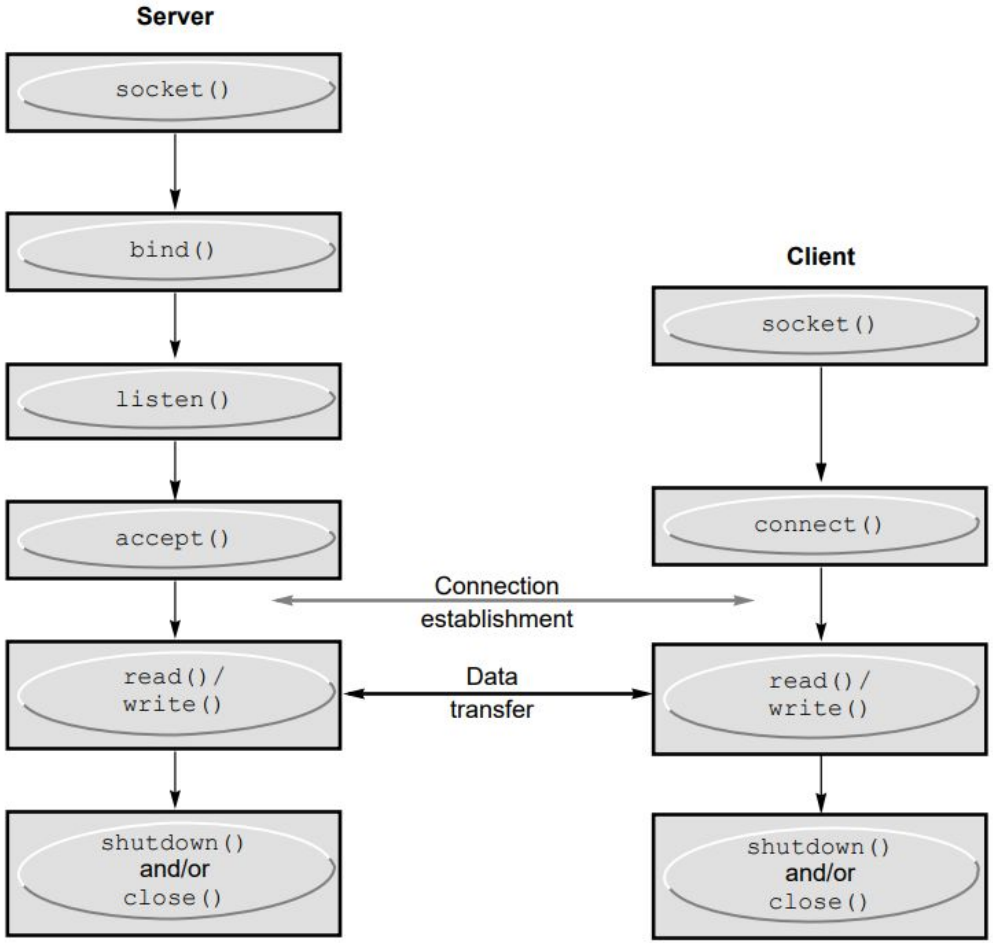
Anybody
wanna talk?



How to set up the connection?

One endpoint has to initiate the connection.

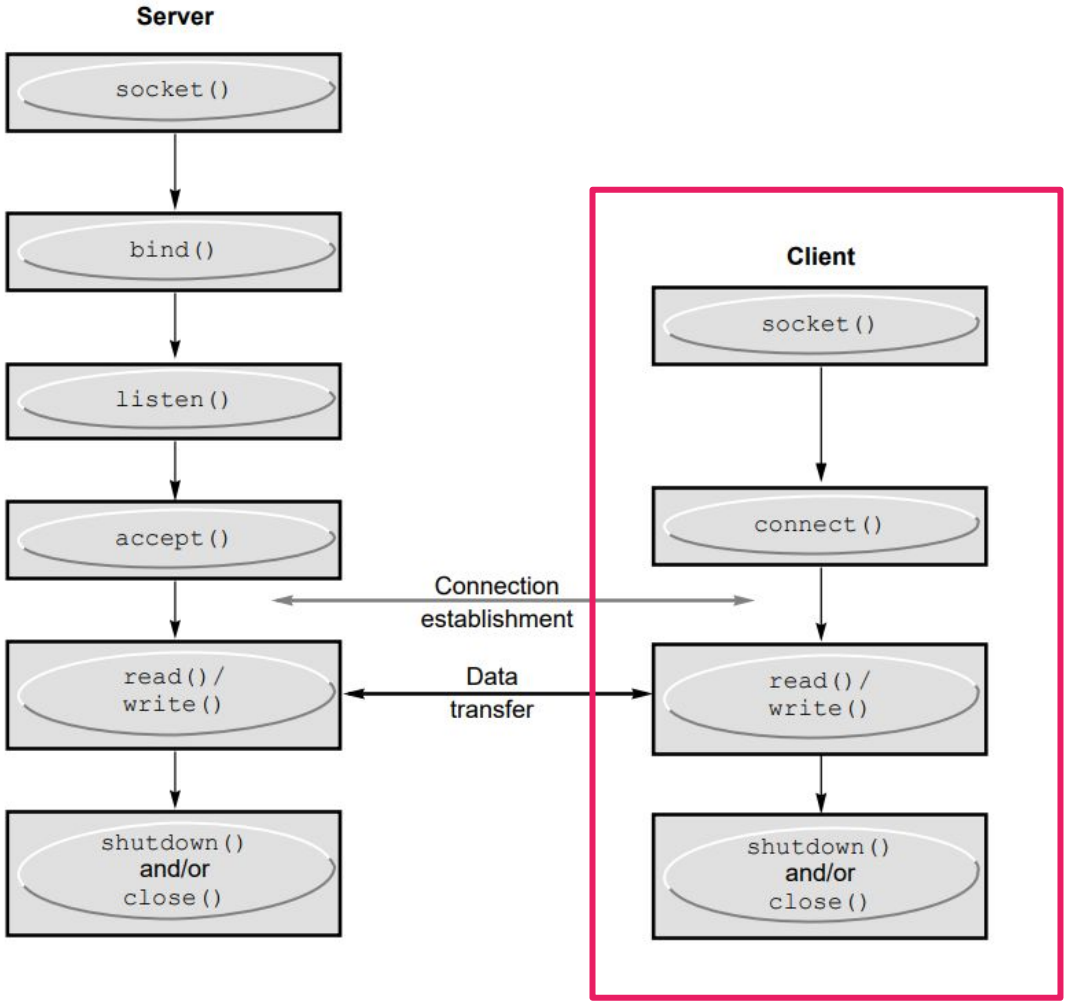




Full socket lifecycle

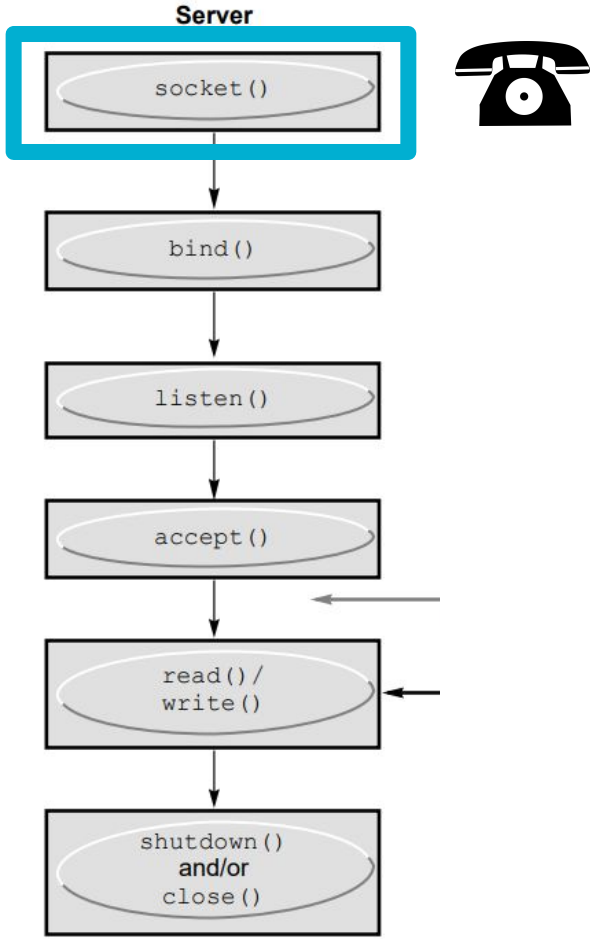
Somewhat analogous to setting up a phone conversion.

Full socket lifecycle



Somewhat analogous to setting up a phone conversion.

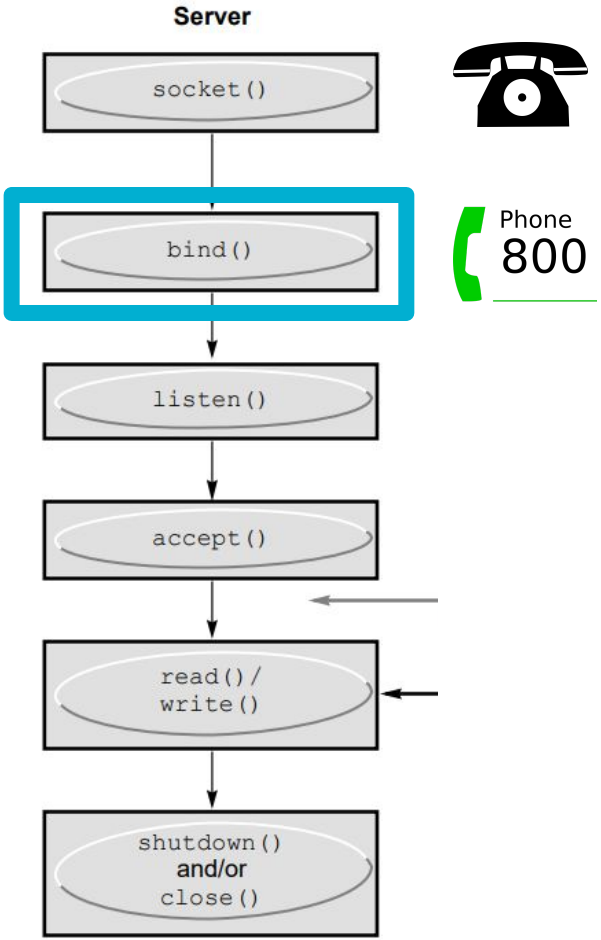
In project 4, **libfs_client.o** implements this functionality



socket(AF_INET, SOCK_STREAM, IPPROTO_IP)

Creates a listener socket.

Kind of like buying a phone.



bind(sock, addr, sizeof(addr))

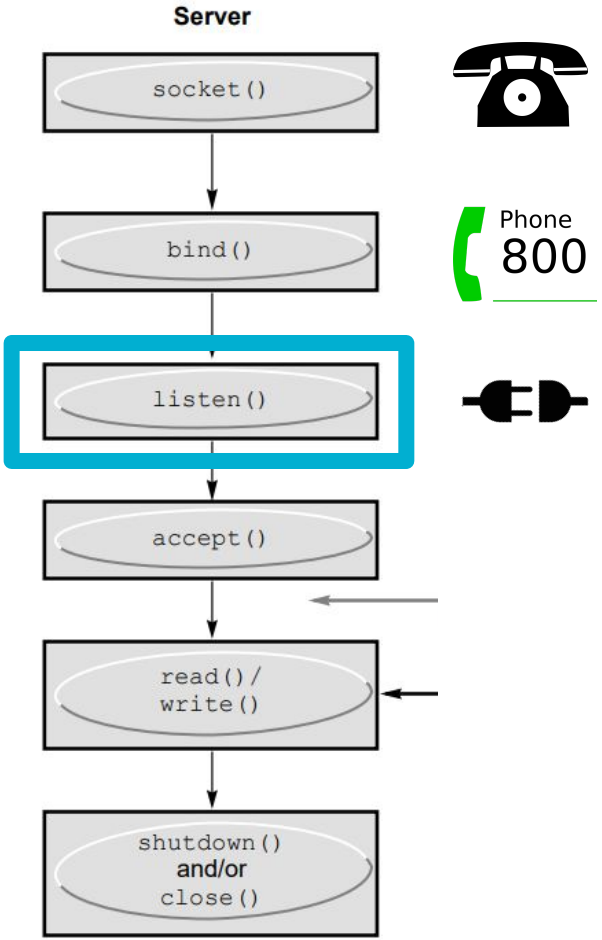
Associates our socket with a port.

Like getting a phone number.

addr is a struct full of parameters that specify what to bind to.

- Port number: addr.**sin_port**

If the port is 0, then the OS chooses a port for you.

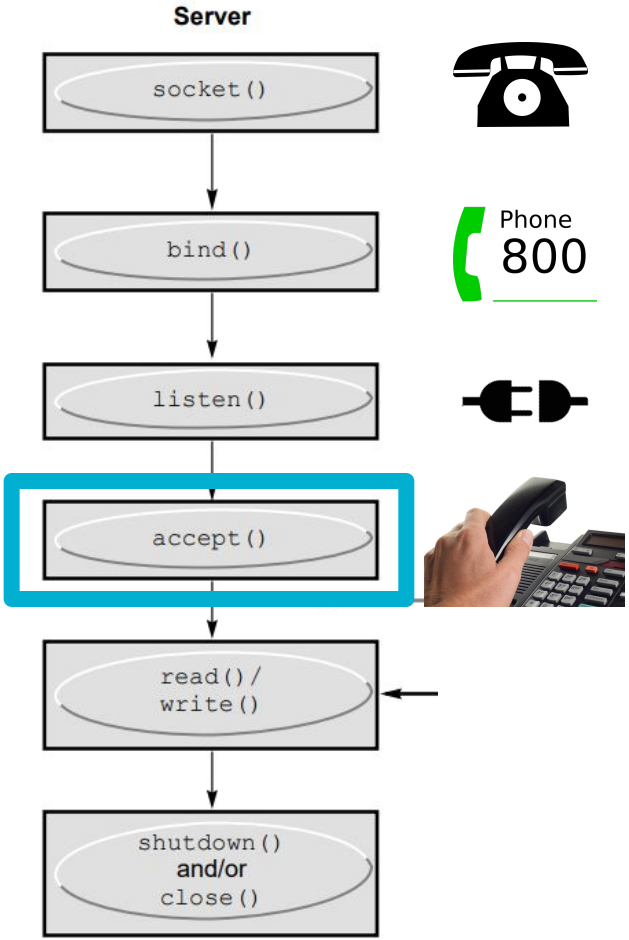


listen(sock, backlog)

Begin listening for connections.

Like plugging the phone in.

Incoming requests wait in a queue of size **backlog**.



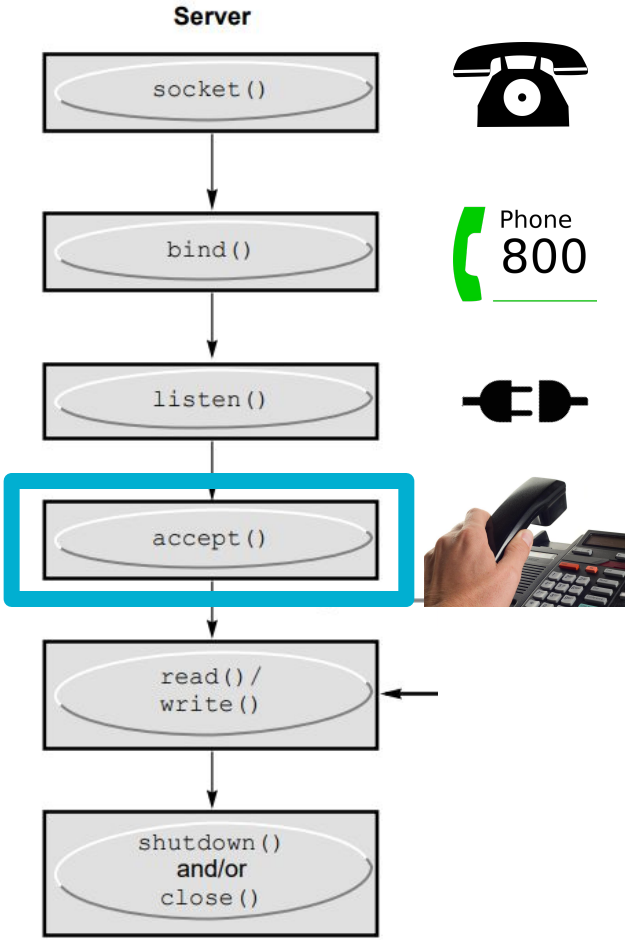
conn = accept(sock, NULL, NULL)

Take a request out of the queue and return a **new** connection socket.

Like answering the phone.

Blocks if no pending connections.

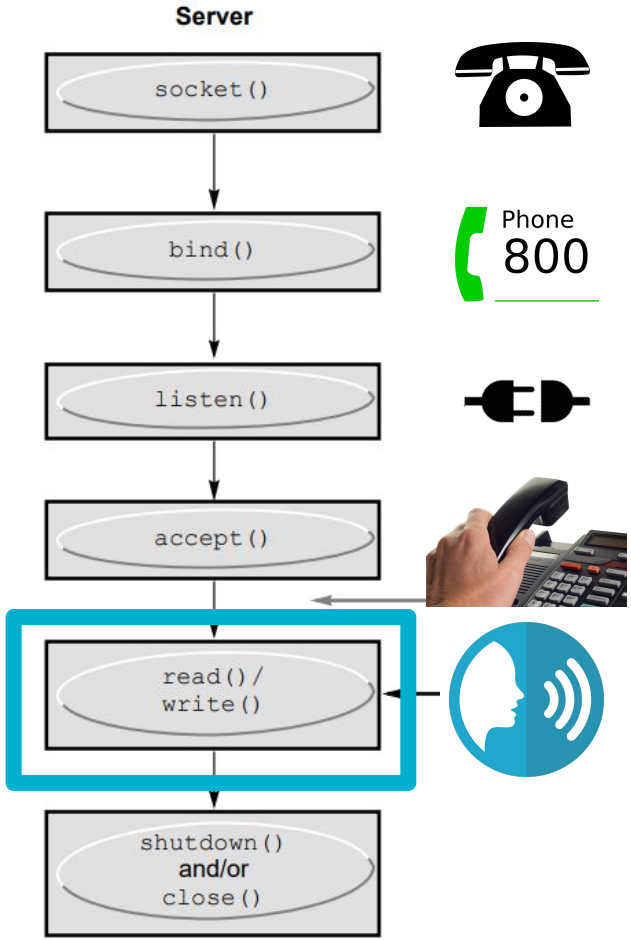
Like waiting for the phone to ring.



conn = accept(sock, NULL, NULL)

Take a request out of the queue and return a **new** connection socket.

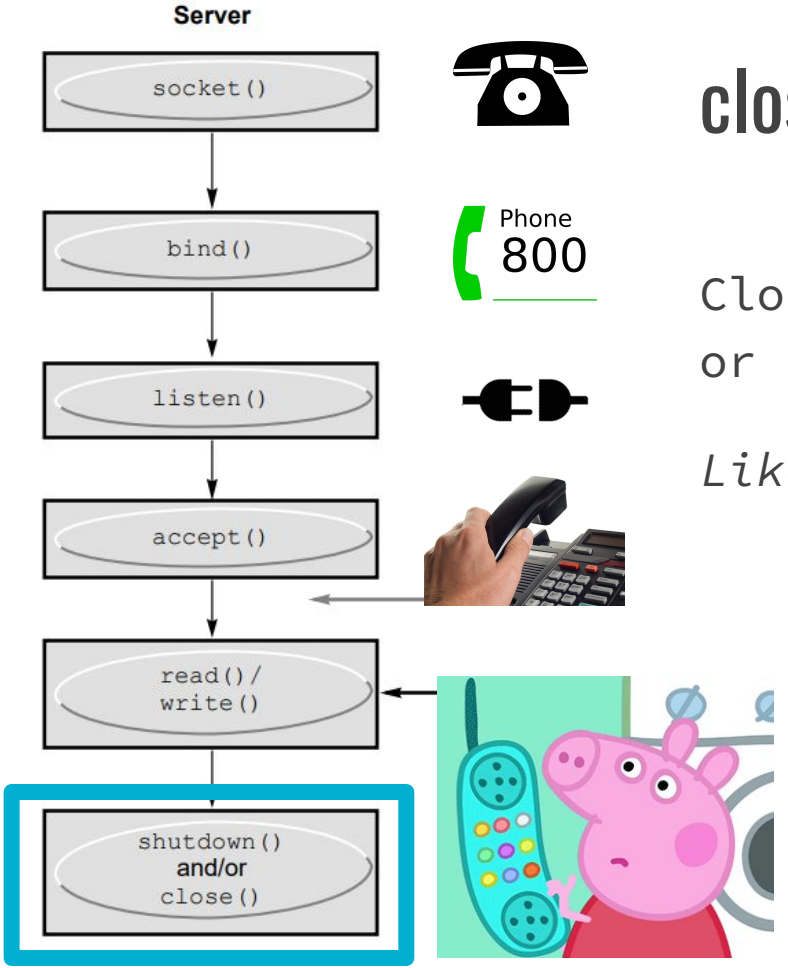
This new socket is what you call send() and recv() on.



send()/recv()

Connection is now established.

May now send() and recv().



close(sock)

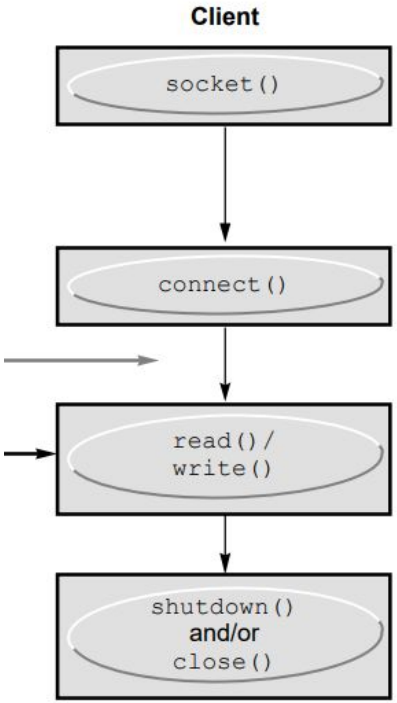
Closes this end of the connection, or closes the listening socket.

Like hanging up.

Client side

Being a client is much simpler.

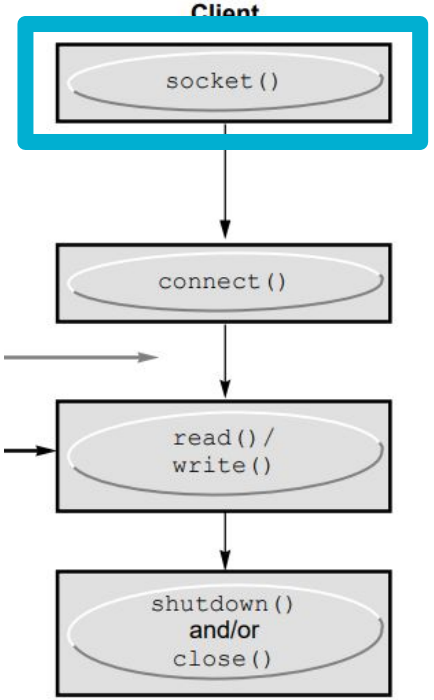
Like calling somebody.



socket(AF_INET, SOCK_STREAM, IPPROTO_IP)

Creates a client socket.

Like buying a cell phone.



Note: no different than creating a listener socket.

All sockets are the same until either listen() or connect() is called.

connect(sock, addr, sizeof(addr))

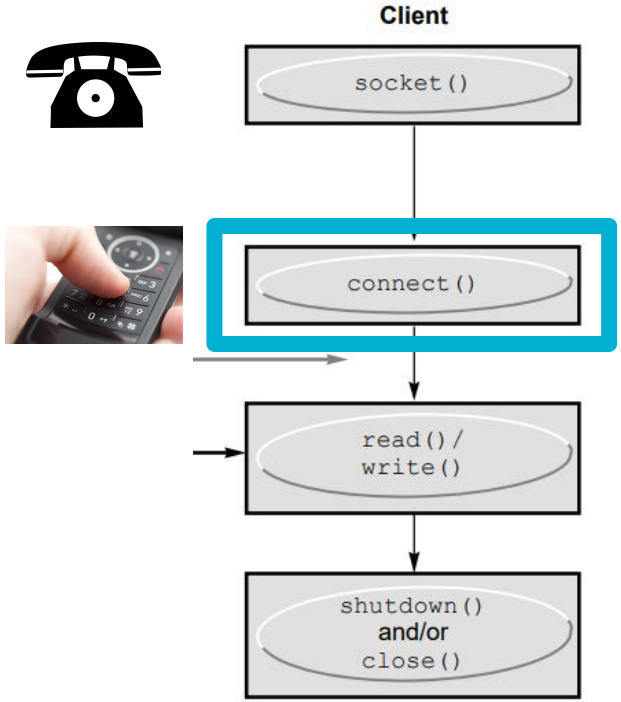
Connect to a remote listener.

Like calling somebody.

addr contains server address and port.

Does **not** return a new connection fd.

Returns 0 on success, -1 on failure.



Note: byte order

Different machines use different byte orders for numbers.

```
int val = 0x0A0B0C0D;
/* little-endian: 0D 0C 0B 0A */
/* big-endian:    0A 0B 0C 0D */
```

There is one single standard network byte order.

Use **ntohs()** / **htons()** to convert to/from network byte order

You must do this when sending numeric types over a network.

https://www.gnu.org/software/libc/manual/html_node/Byte-Order.html

Programming Best Practice: **Always Check Return Value**

- Always check the return values of functions
  - Regardless of what you expect the function to return
- Networking functions can fail
  - Always check for these cases
- What happens when we don’t check?
  - Undefined behavior
  - For example
    - If socket() returns -1, does it make sense to call bind()?
    - If recv() returns -1, does it make sense to call recv() again?
- Use assertions if you expect something to never fail
  - For example
    - int close_res = close(fd);
    - assert(close_res == 0);

better practice: use RAII and let an exception be thrown

Testing network code: netcat

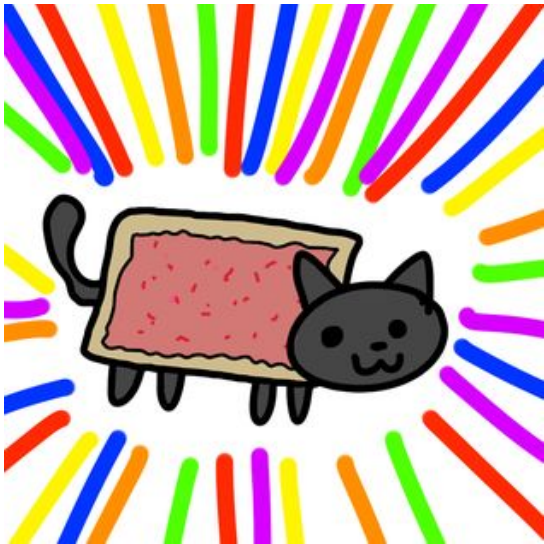
---

netcat is like cat over a network

Original executable named nc, but newer ncat is nicer to use

Install ncat (package probably named nmap)

man ncat to see usage



Socket programming lab exercise

Write a simple client and server, where the client sends a string to the server and the server outputs the string using cout.

https://os.eecs.umich.edu/lab/sockets.html
Beej's Guide to Network Programming

Testing network code: netcat

Try it out! Use ncat to send messages to us :)

We'll run ncat listening for incoming connections

Connect and print stuff on our terminal!

ncat [hostname] [port]

Questions?